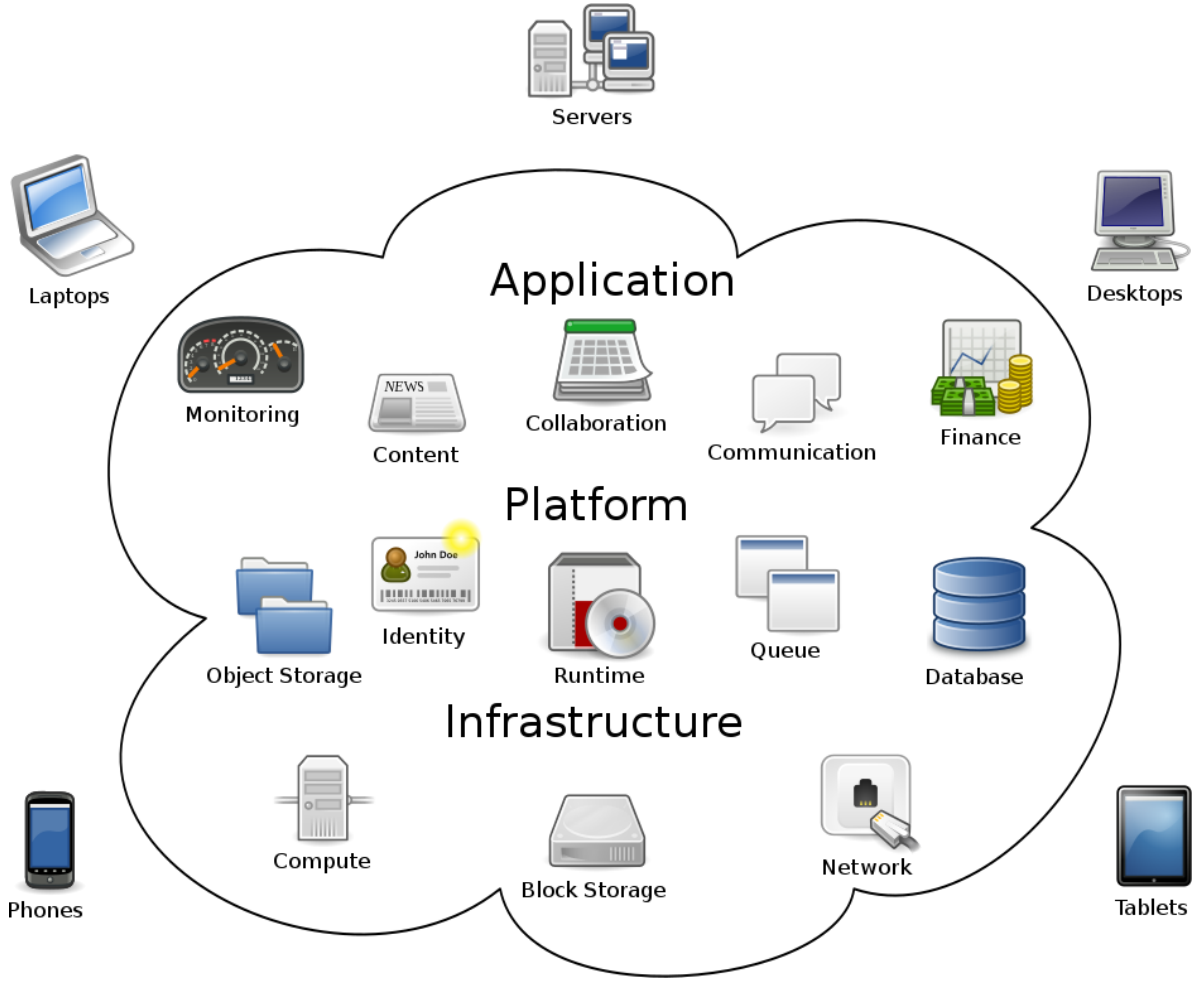
WAN Overview

Course: Networking Principles in Practice – Cloud Networking
Module: WAN



University of Colorado **Boulder**

Cloud – as a Service models



Datacenter

Nederlands



Inside a Google data center

https://www.youtube.com/watch?v=XZmGGAbHqa0&ab_channel=GoogleWorkspace

Regions

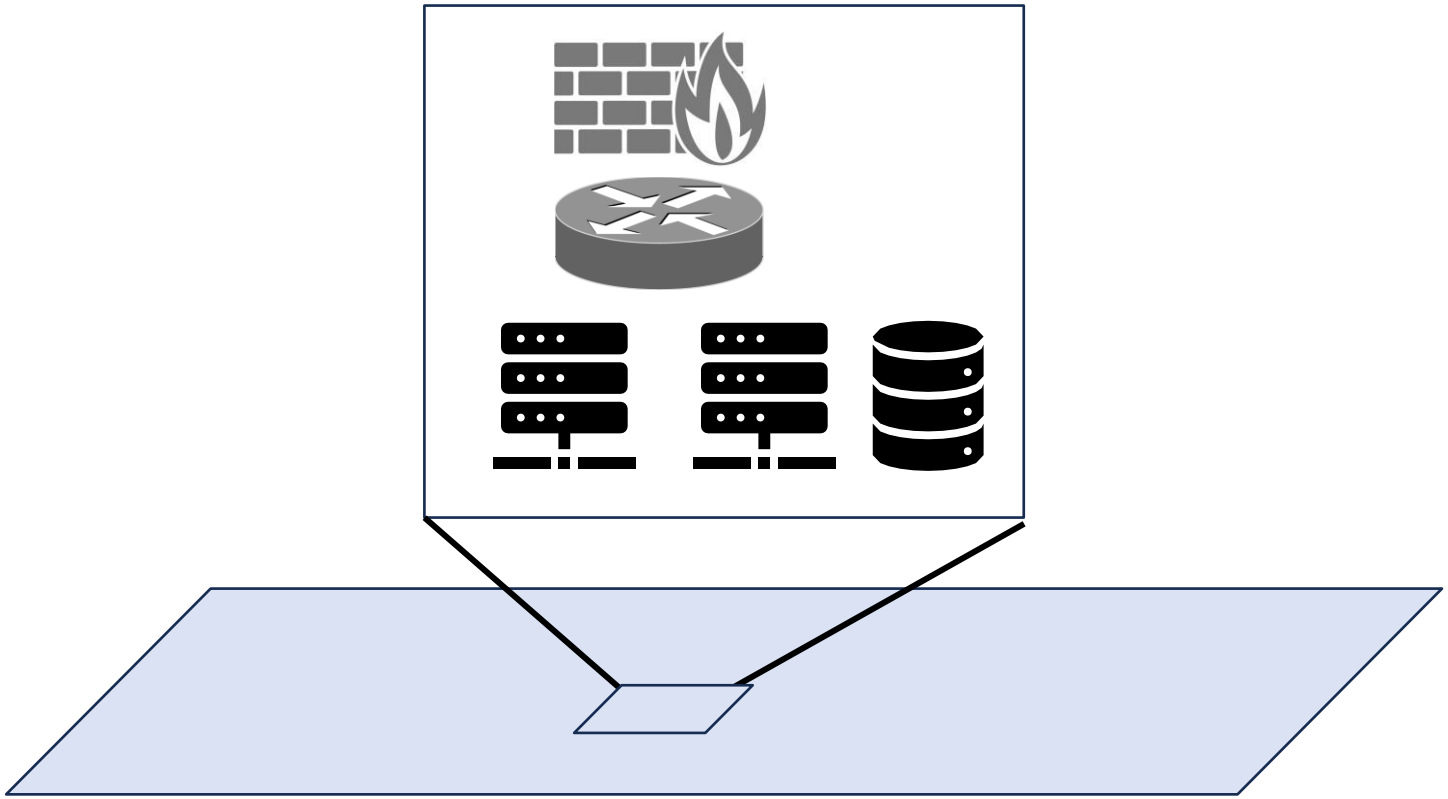
- Correlate to locations of Google Datacenters
- Named based on location (e.g., us-west1, asia-east1)
- gcloud compute regions list



https://cloud.google.com/about/locations#lightbox-regions-map

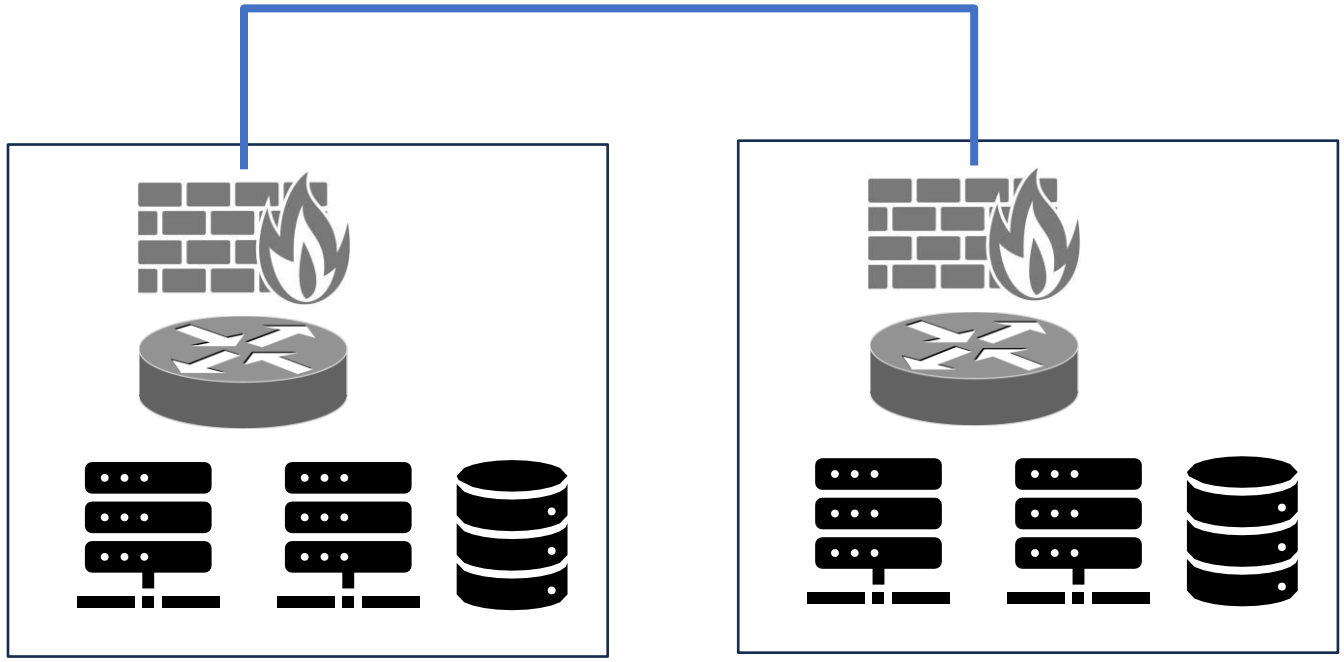
Virtual Private Cloud (VPC)

- Virtual Private Cloud – effectively your own private cloud running on top of the providers physical infrastructure

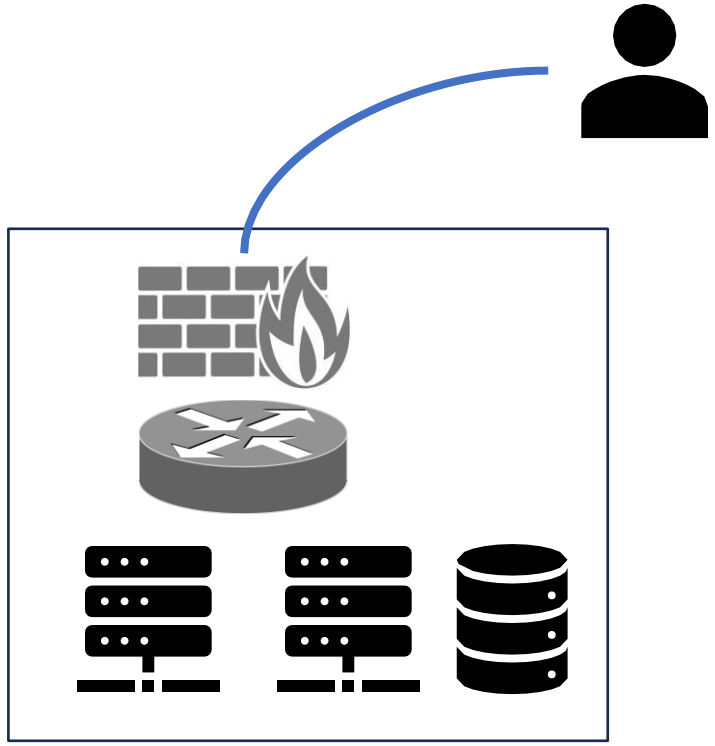


Communication Beyond the VPC Bounds

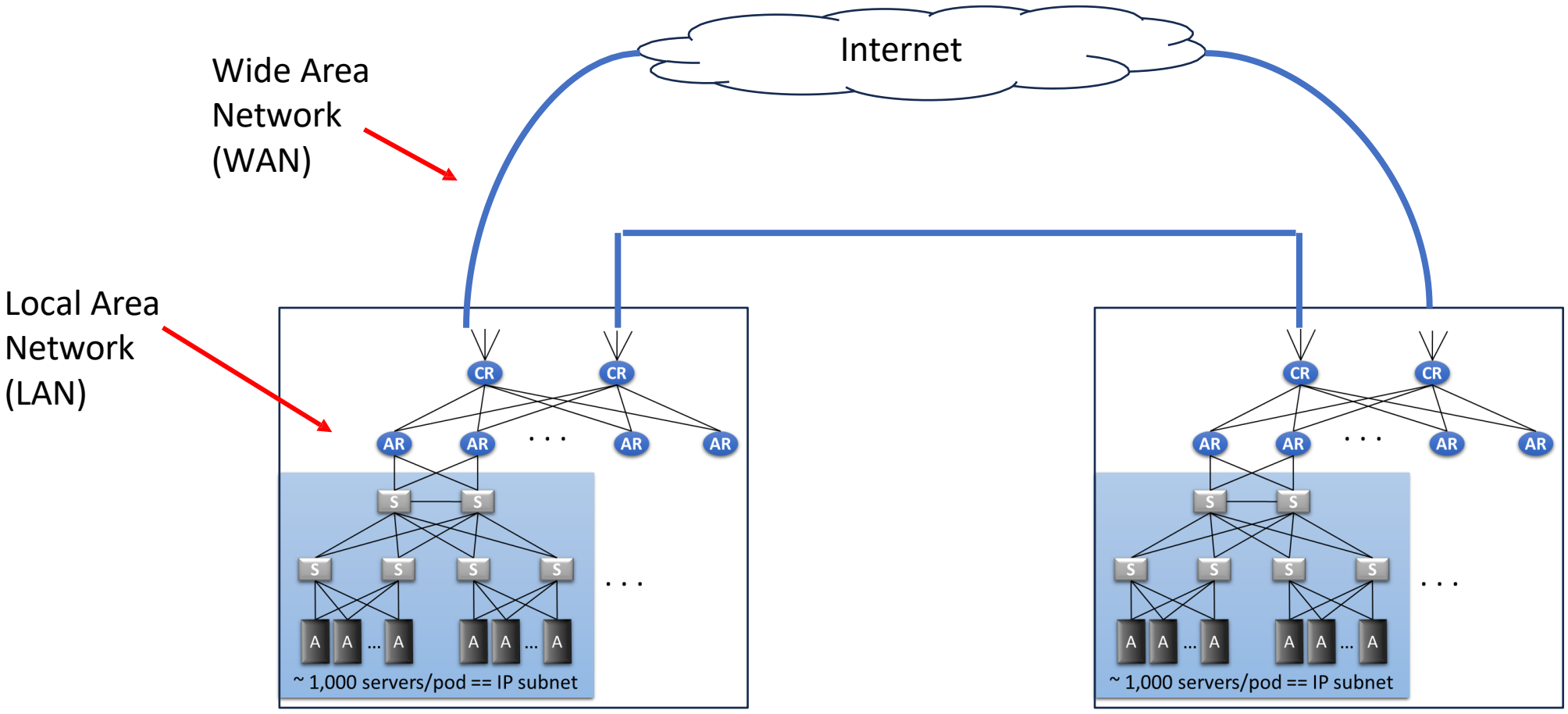
VPC to VPC



VPC to end user

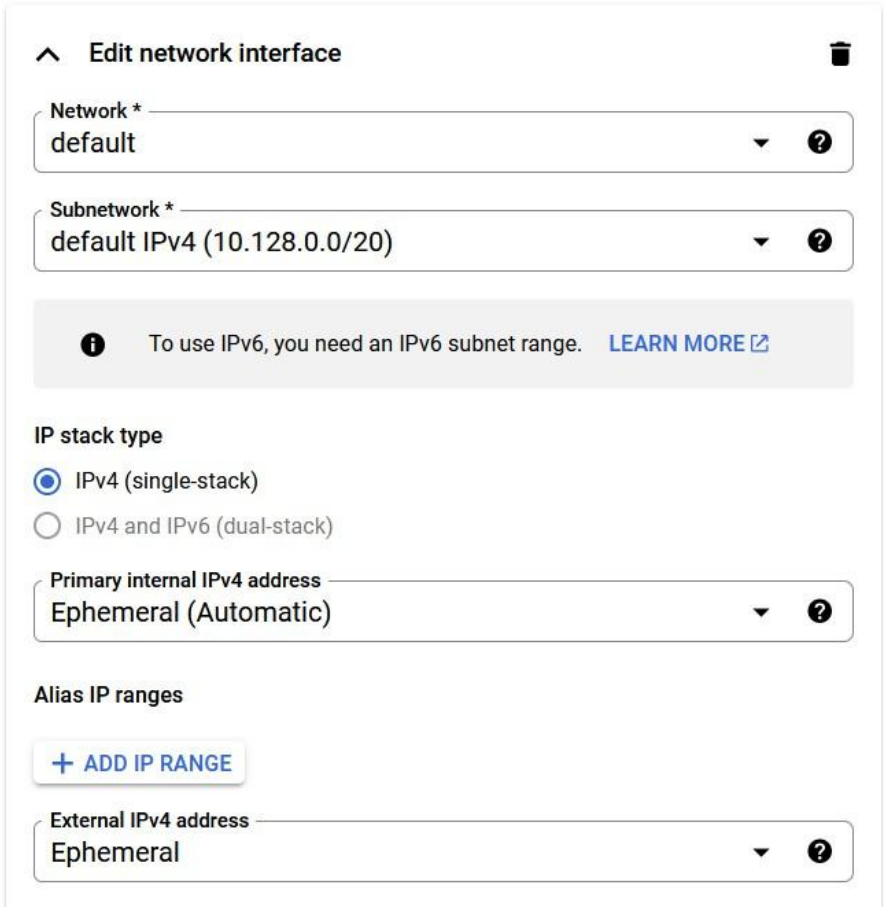


WAN vs LAN


Wide Area Network (WAN)
Local Area Network (LAN)
Internet
CR
AR
S
A
~ 1,000 servers/pod == IP subnet

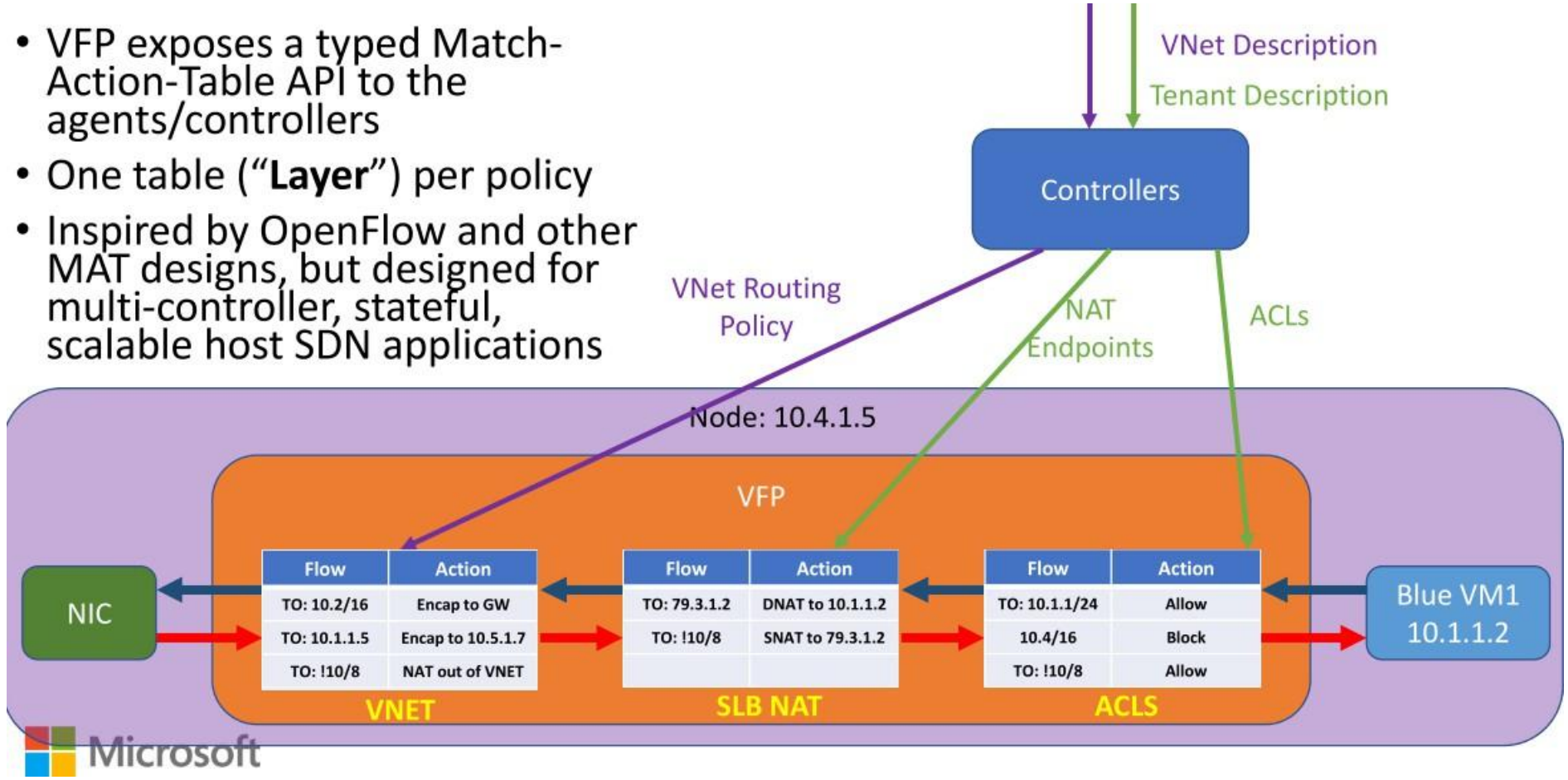
Public / Private IP addresses

- RFC 1918
  - 10.0.0.0 - 10.255.255.255 (10/8 prefix)
  - 172.16.0.0 - 172.31.255.255 (172.16/12 prefix)
  - 192.168.0.0 - 192.168.255.255 (192.168/16 prefix)
- Internal IP addresses – within subnet which is within VPC
- Recall: Creating VM (pic on right)
  - If you run ip addr – note, you don't see the external.



Recall: AccelNet and Andromeda

- VFP exposes a typed Match-Action-Table API to the agents/controllers
- One table ("**Layer**") per policy
- Inspired by OpenFlow and other MAT designs, but designed for multi-controller, stateful, scalable host SDN applications



Microsoft

Communicating with public IP addresses

- Google IP addresses, will traverse their backbone
- Otherwise, will use a peering to an ISP

Communicating with private IPs

Two key mechanisms we'll cover for enabling communication with the private IP addresses

- VPC Peering
- Cloud NAT

Outline of module

- VPC Peering
- Cloud NAT
- SWAN
- Espresso
- Lab overview



University of Colorado **Boulder**

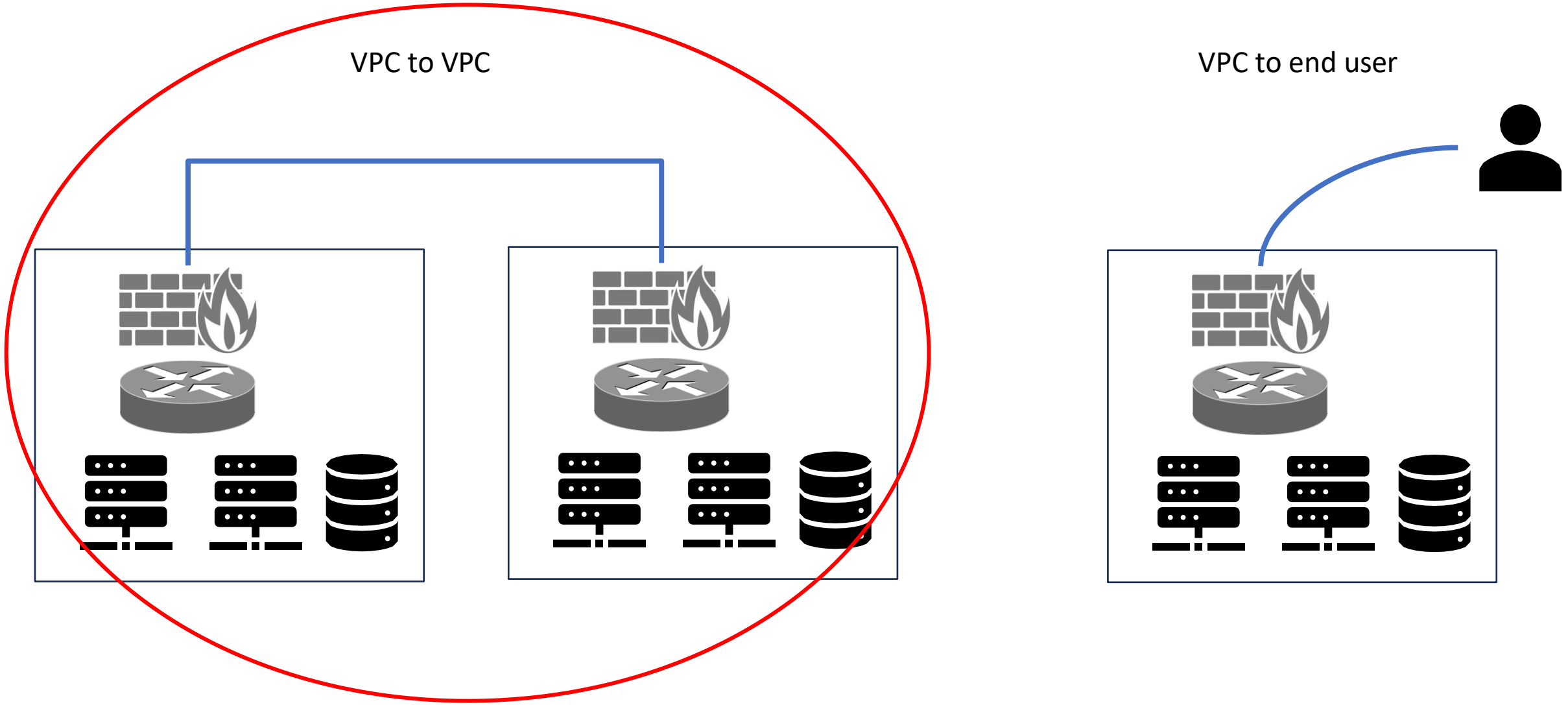
VPC Peering

Course: Networking Principles in Practice – Cloud Networking
Module: WAN



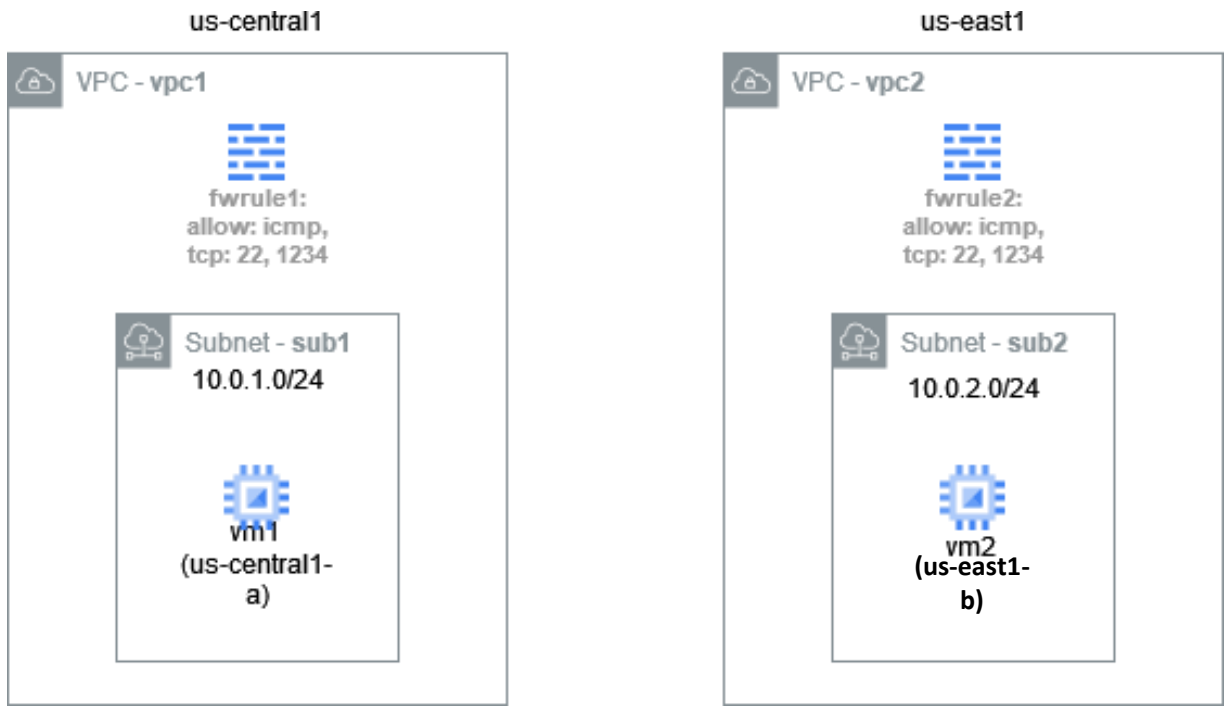
University of Colorado **Boulder**

Communication Beyond the VPC Bounds



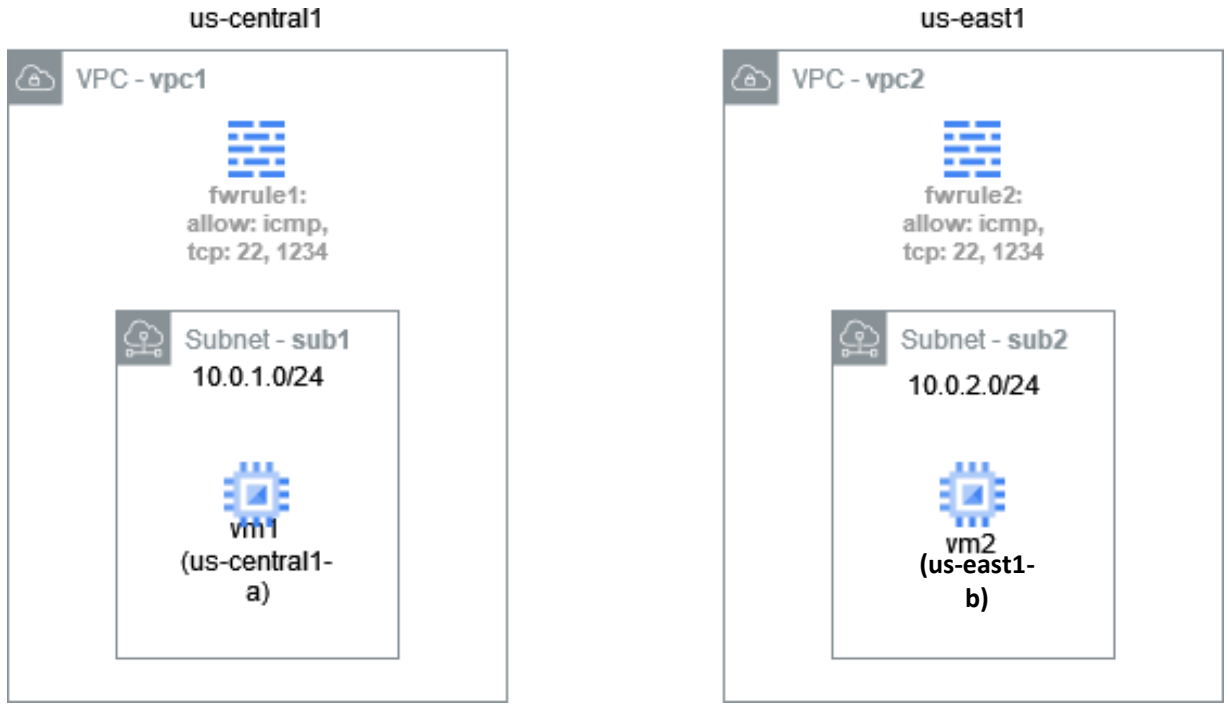
Setup

- Create this in console
- Check box that says “global” route exchange



Goal: Communicate with Each Other via Private IPs

- SSH into VM1 and VM2
  - ip addr
  - ping private IP
- Optional test: create another subnet within vpc1

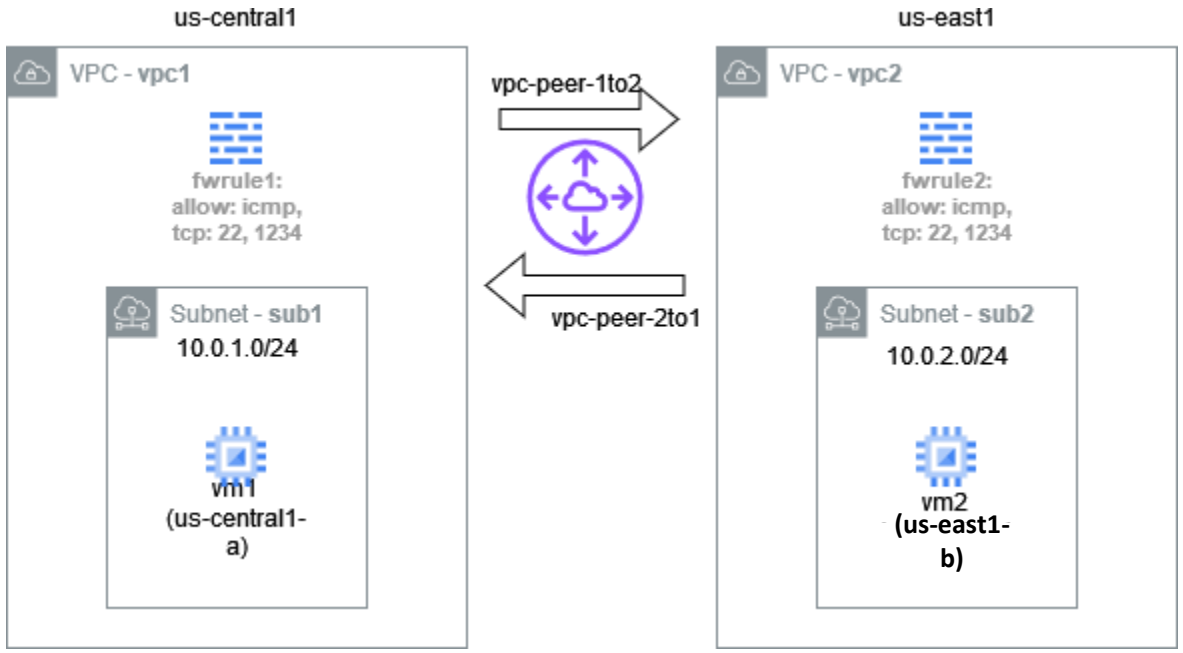


VPC Peering

- https://cloud.google.com/vpc/docs/vpc-peering
- Sets up a private connection between two VPCs
  - Must create one peering in each direction
- Performs route exchange between VPCs
- Can’t have subnets with overlapping address space
  e.g., vpc1.sub1 with 10.0.0.0/24 and vpc2.sub2 with 10.0.0.0/24

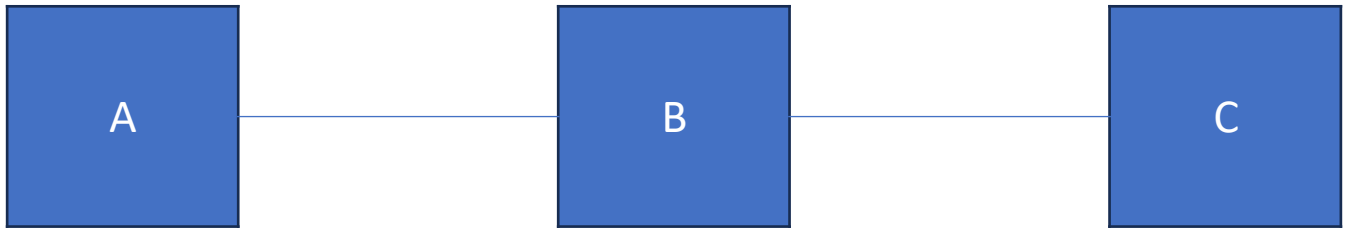
Demo in console

- Add Peering vpc1 to vpc2 (and 2 to 1)
- From VM1 ping VM2
- Show routes in each peering network.
- Add a subnet
- Note about hybrid infrastructure



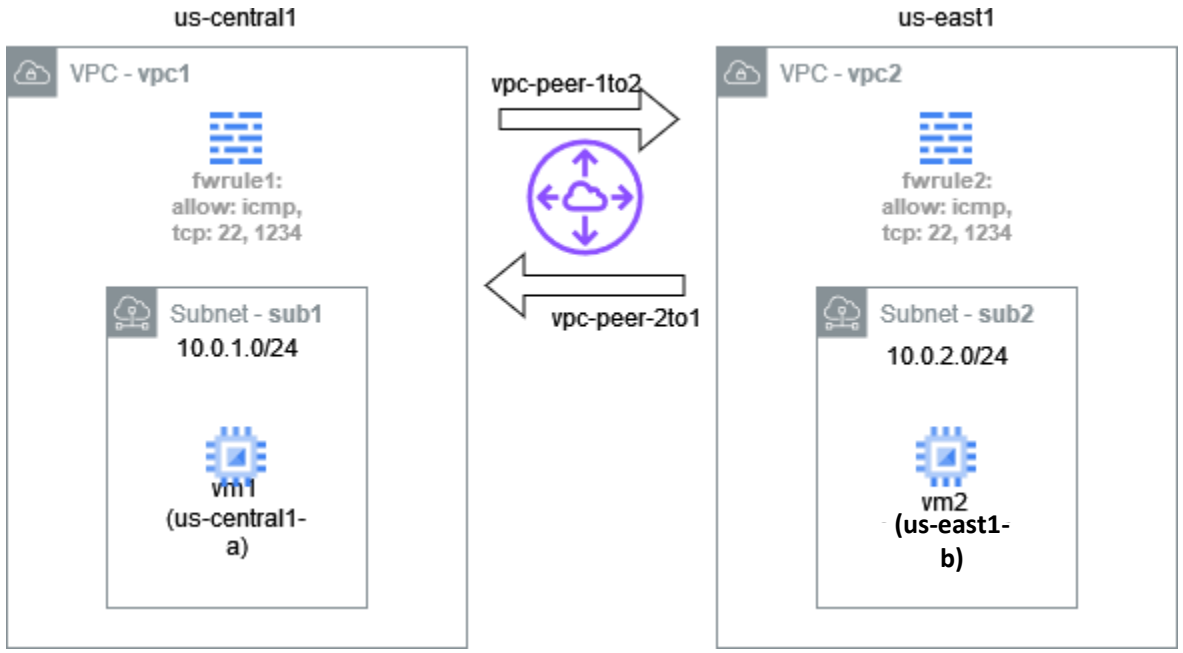
Non-Transitive

- Peering between A and B, and B and C
- B would be a transitive VPC
- GCP does not support transitive VPCs
  - A does not receive routes of C



Firewall

- vpc1 to vpc2 traffic would still traverse the firewall



Terraform: Recall VPC / subnet

```
resource "google_compute_subnetwork" “vpc1-sub1" {
   name          = “vpc1-sub1“
   ip_cidr_range = "10.2.0.0/16“
   region        = "us-central1“
   network       = google_compute_network.vpc1.id
   secondary_ip_range {
      range_name    = " vpc1-subnetwork -range “
      ip_cidr_range = "192.168.10.0/24"
   }
}
resource "google_compute_network" “vpc1" {
   name                    = “vpc1“
   auto_create_subnetworks = false
}
```

Terraform: Network Peering

```
resource "google_compute_network_peering" "peering1" {
  name = "peering1"
  network = google_compute_network.vpc1.self_link
  peer_network = google_compute_network.vpc2.self_link
}

resource "google_compute_network_peering" "peering2" {
  name = "peering2"
  network = google_compute_network.vpc2.self_link
  peer_network = google_compute_network.vpc1.self_link
}
```

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_network_peering



University of Colorado **Boulder**

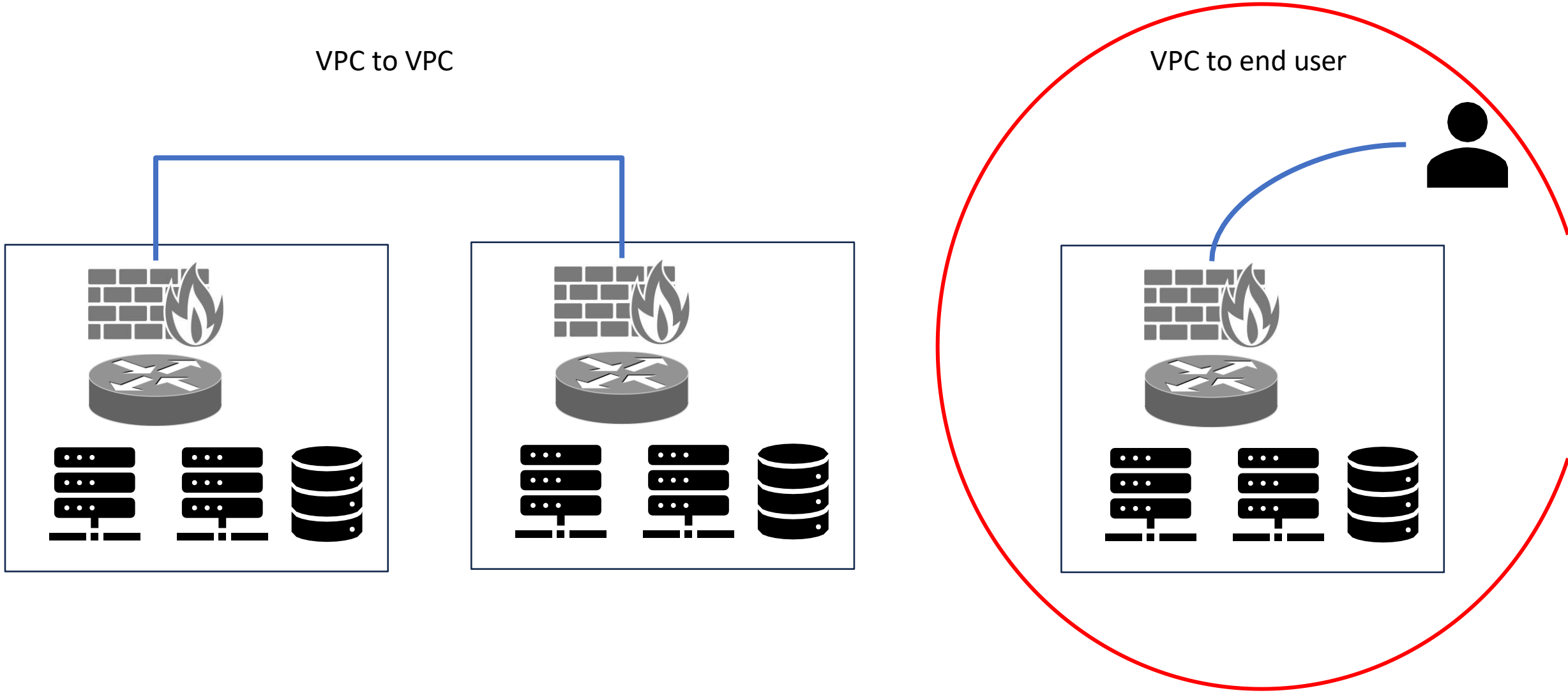
Cloud NAT

Course: Networking Principles in Practice – Cloud Networking
Module: WAN



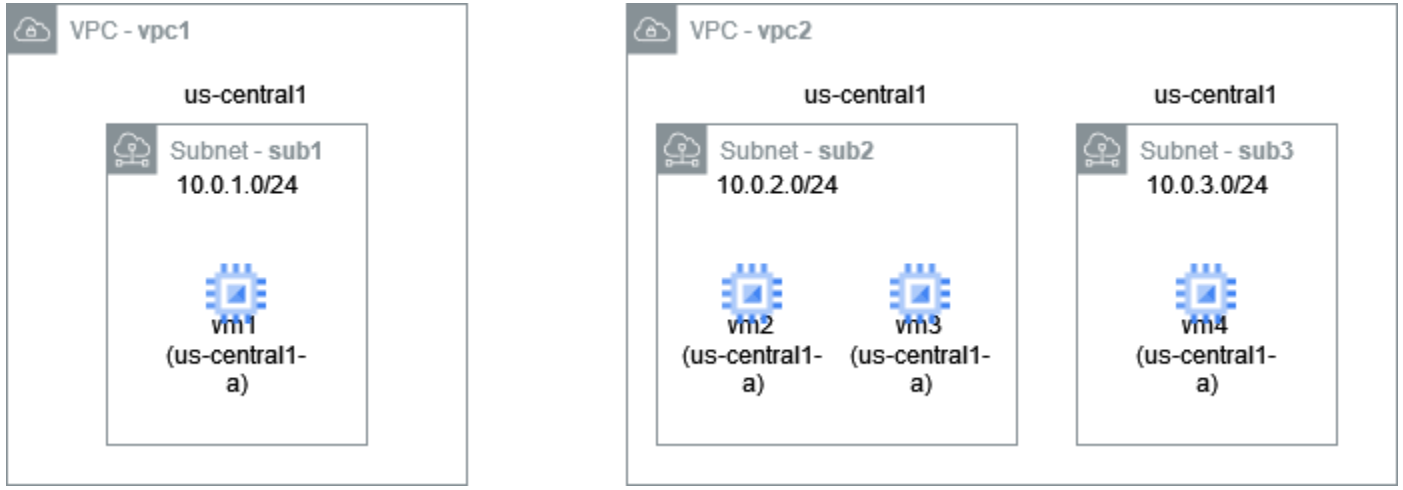
University of Colorado **Boulder**

Communication Beyond the VPC Bounds



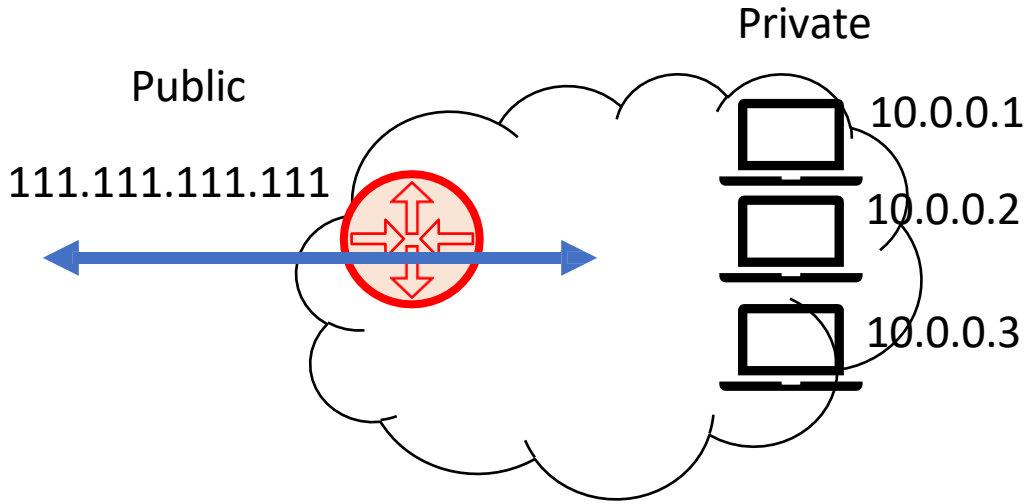
Setup

- Recall this setup from module 2
- Vm1 and vm2 have external IP, vm3 and vm4 did not
- We'll create just vpc1, sub2, and vm1 (without public IP) in console
  - ssh into vm1 - ping google.com, ping 8.8.8.8



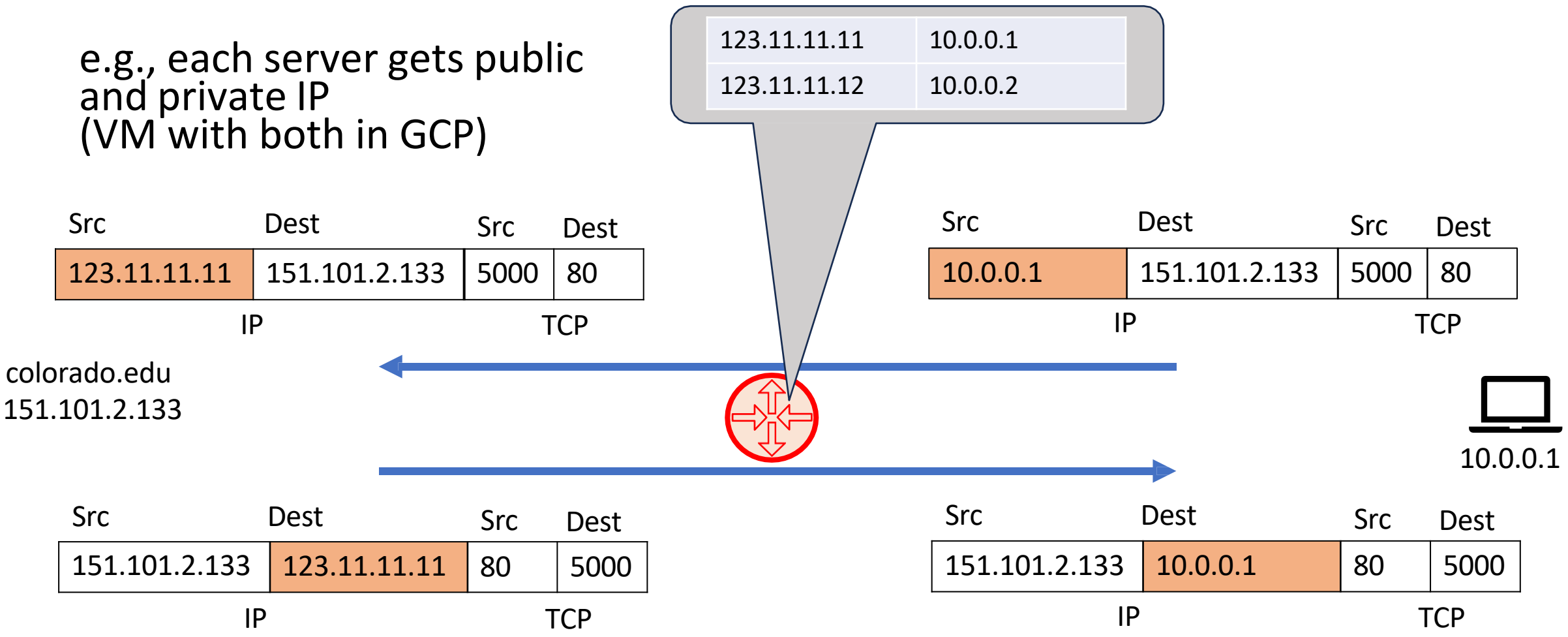
Review: Address translation

- Actions can also manipulate the address
  - source/destination IP addresses
  - source/destination ports
- Example: use private addressing on the LAN, with a single shared public IP address.
  - Packets destined to public address are translated to a private one



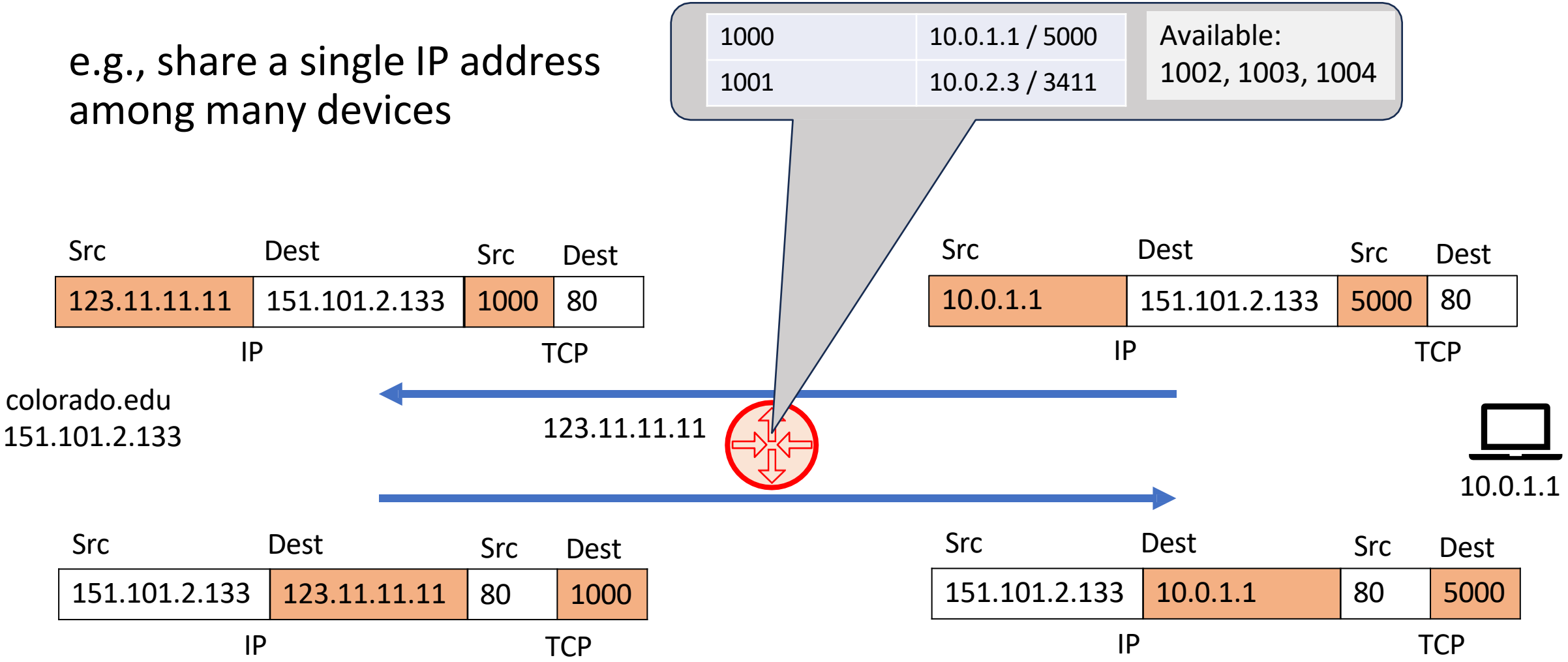
Use Case 1: Static Mapping - Internal to External

e.g., each server gets public and private IP
(VM with both in GCP)



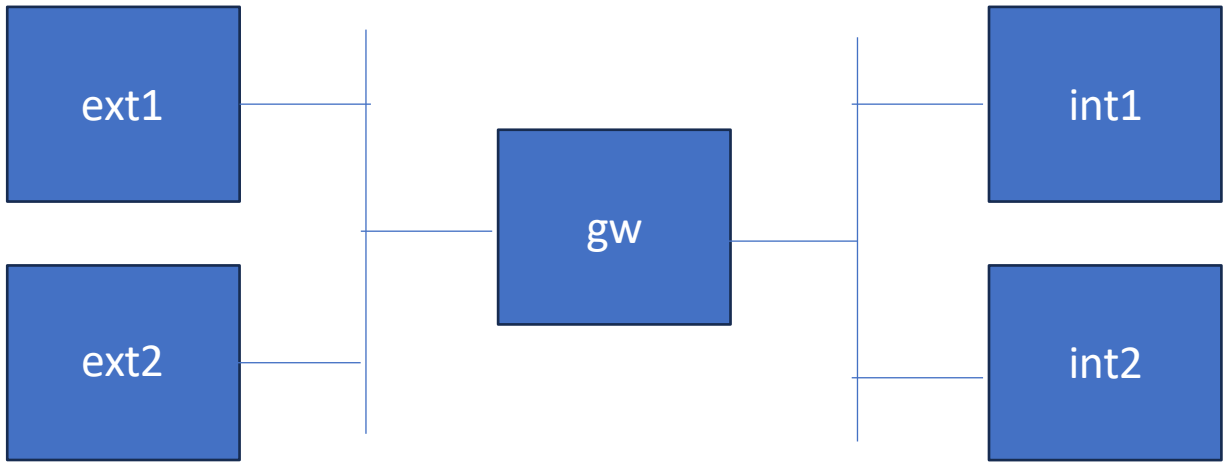
Use Case 2: Dynamic Mapping Single External IP (dynamically select from available TCP ports)

e.g., share a single IP address among many devices



NAT in Linux Review

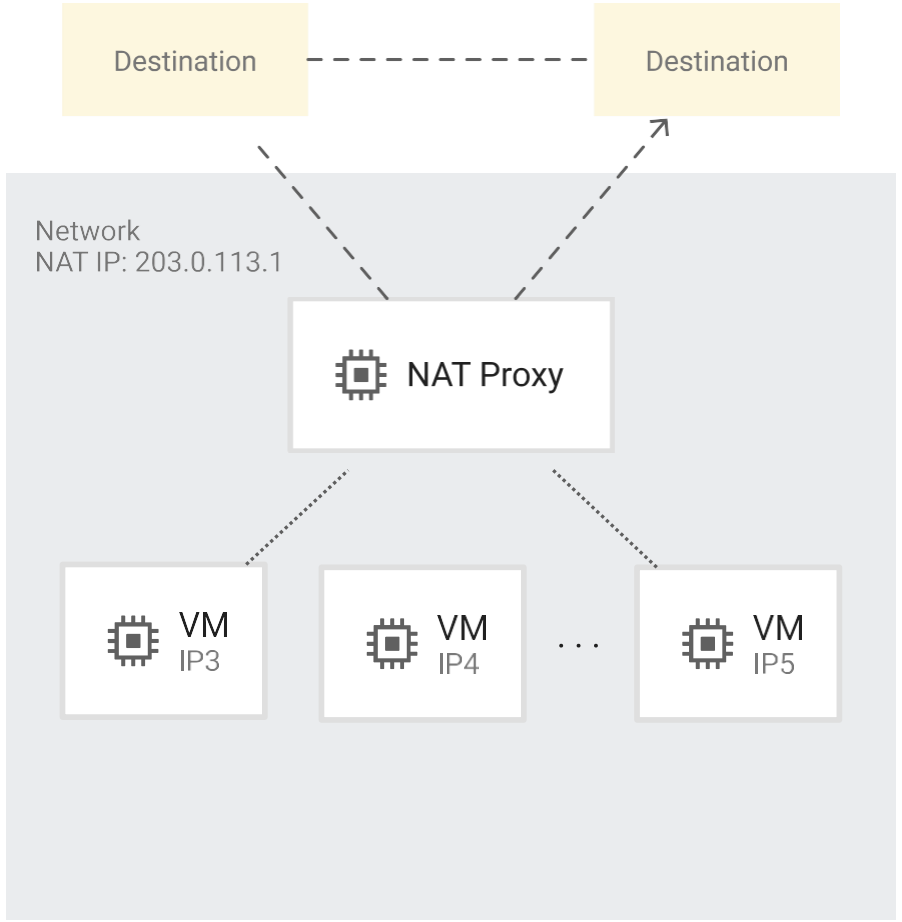
- iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE
  iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
- iptables -t nat -A POSTROUTING -o eht0 -s 192.168.1.3 -j SNAT --to 1.2.3.4



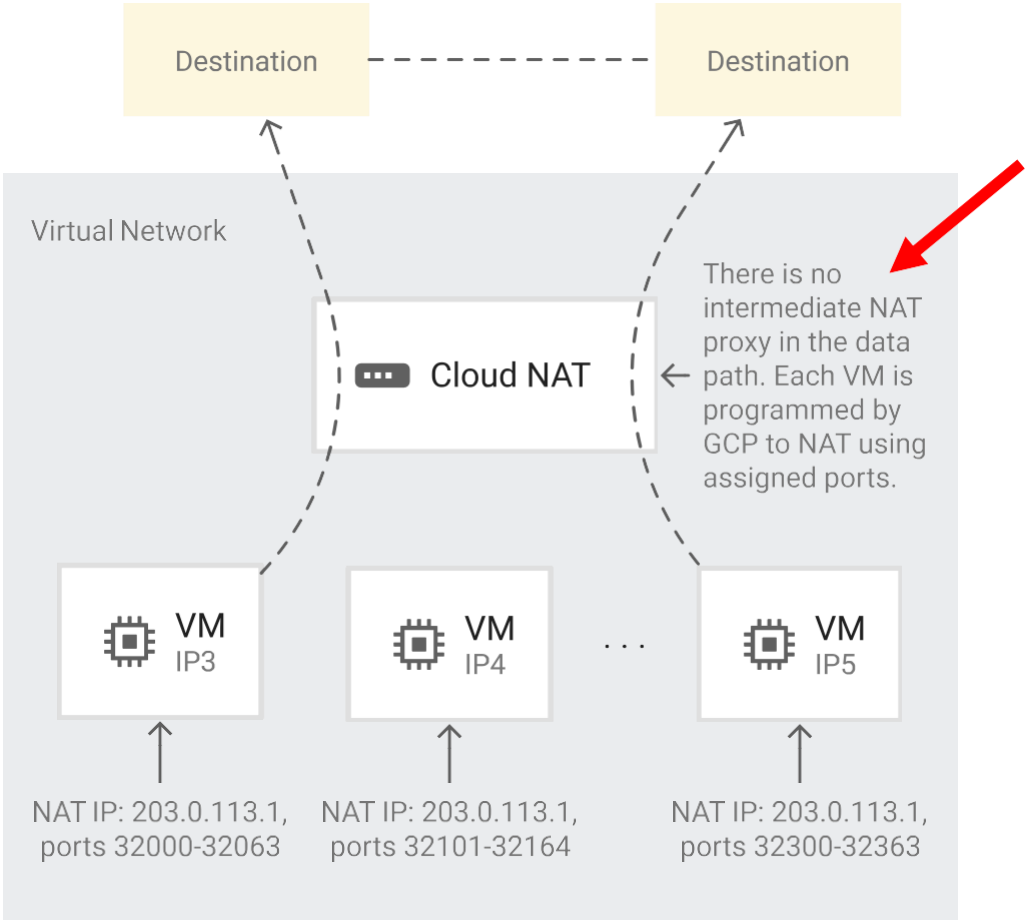
Cloud NAT

- Google service that provides access to external network for VMs that don’t have external IP address.
- Effectively provides this same concept for a VPC

Cloud NAT Realization



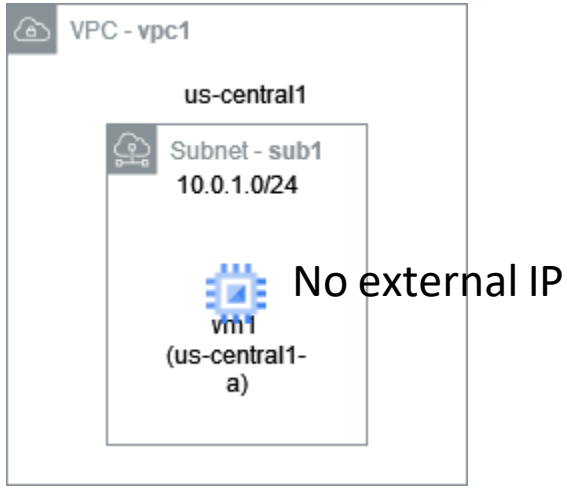
1. Typical NAT Proxies



2. Google Cloud NAT

Instantiating in Console

- Walkthrough in Console: Add NAT to setup
- Note: Need a Cloud Router
- ssh into VM1 again and ping google.com – should work now.



Terraform

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_router_nat

```
resource "google_compute_router" "router" {
  name    = "my-router"
  region  = google_compute_subnetwork.subnet.region
  network = google_compute_network.net.id

  bgp {
    asn = 64514
  }
}

resource "google_compute_router_nat" "nat" {
  name                               = "my-router-nat"
  router                             = google_compute_router.router.name
  region                             = google_compute_router.router.region
  nat_ip_allocate_option             = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"

  log_config {
    enable = true
    filter = "ERRORS_ONLY"
  }
}
```



University of Colorado **Boulder**

Software Driven WAN

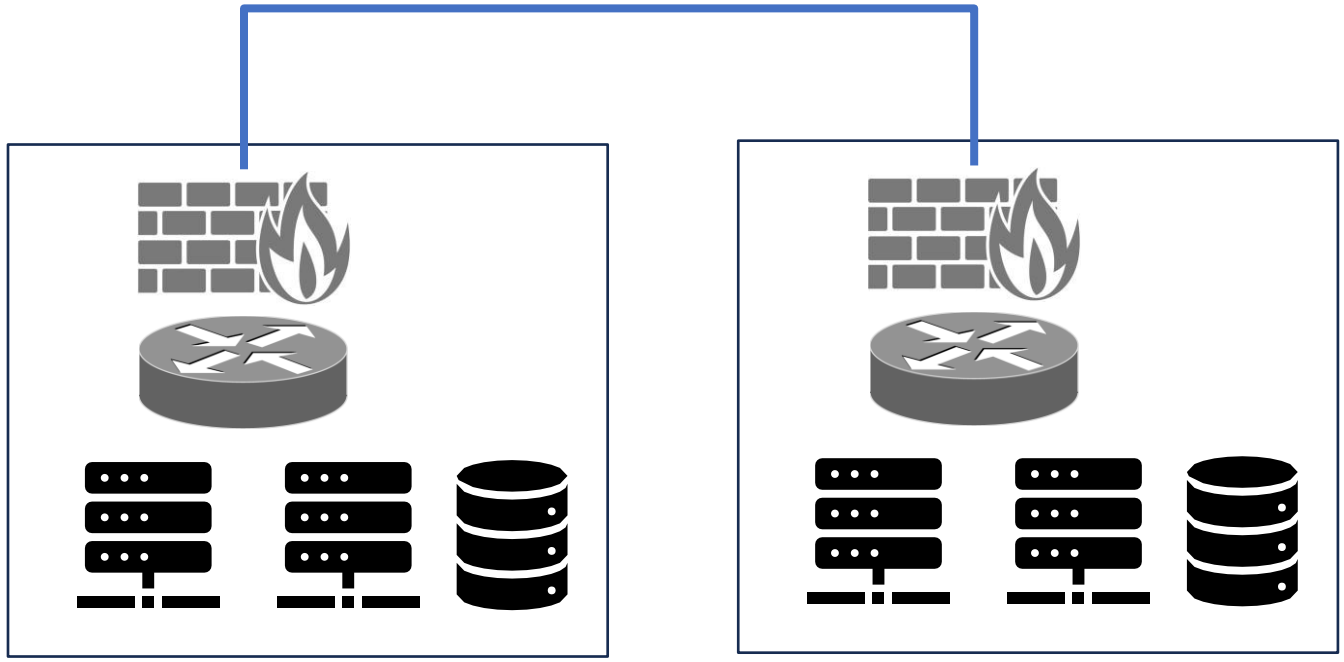
Course: Networking Principles in Practice – Cloud Networking
Module: WAN



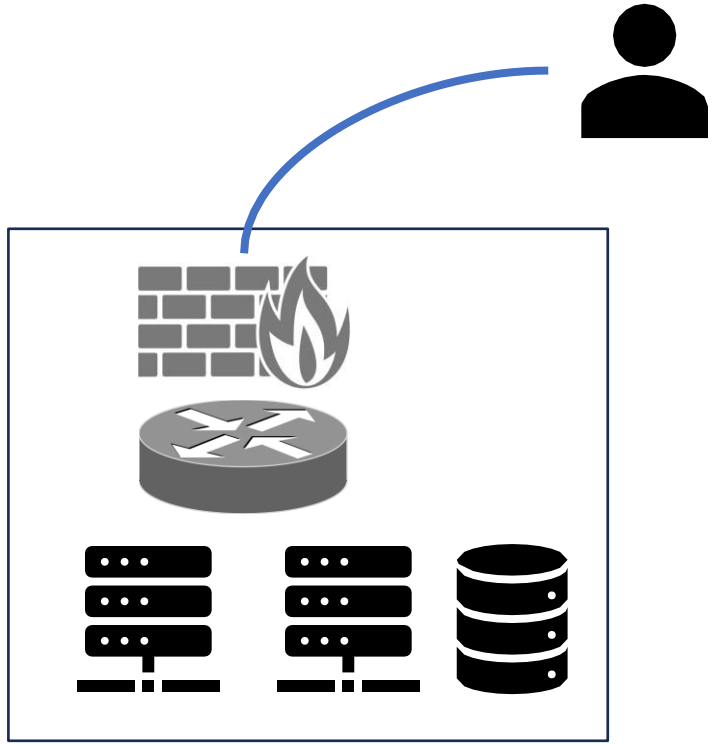
University of Colorado **Boulder**

Communication Beyond the VPC Bounds

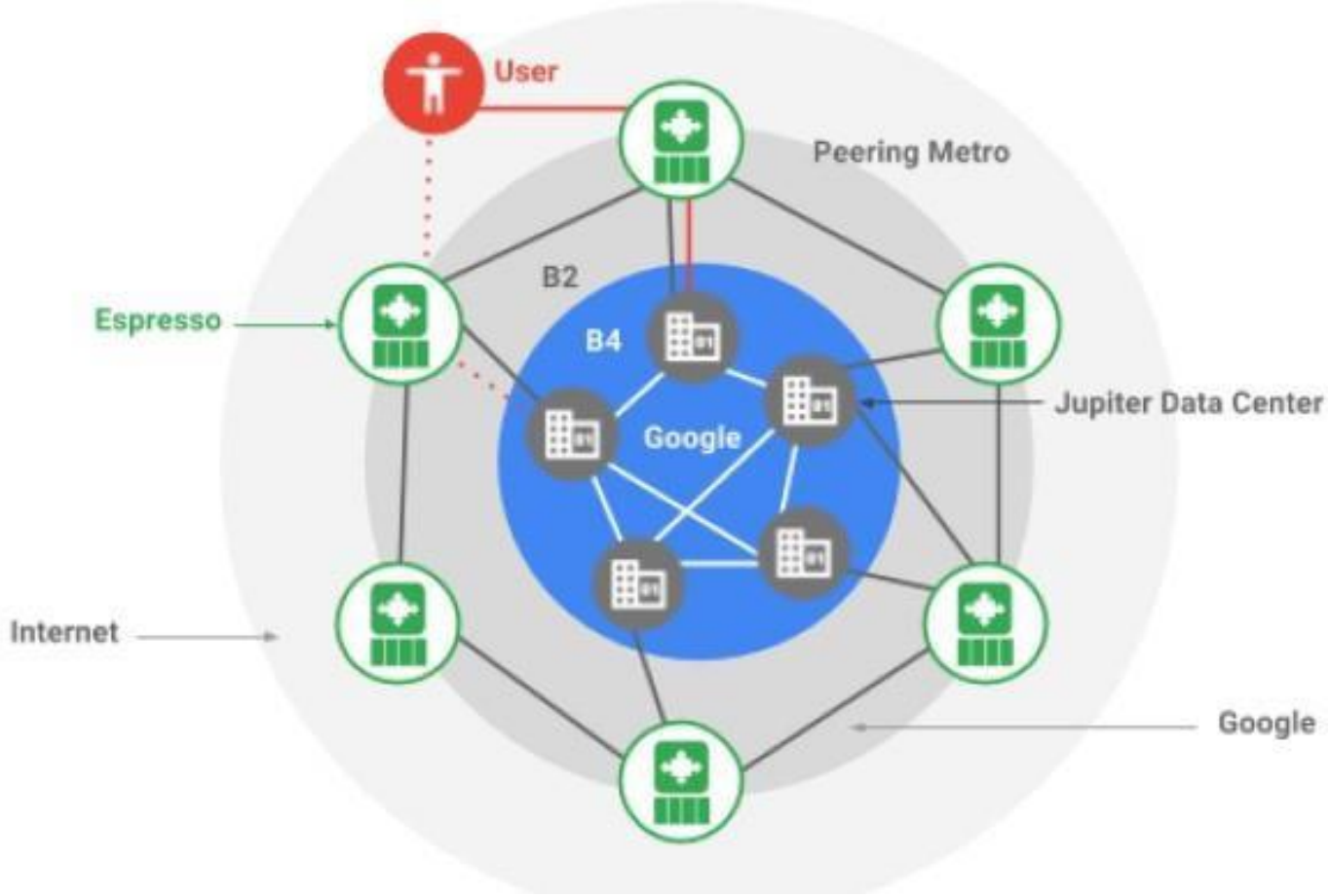
VPC to VPC



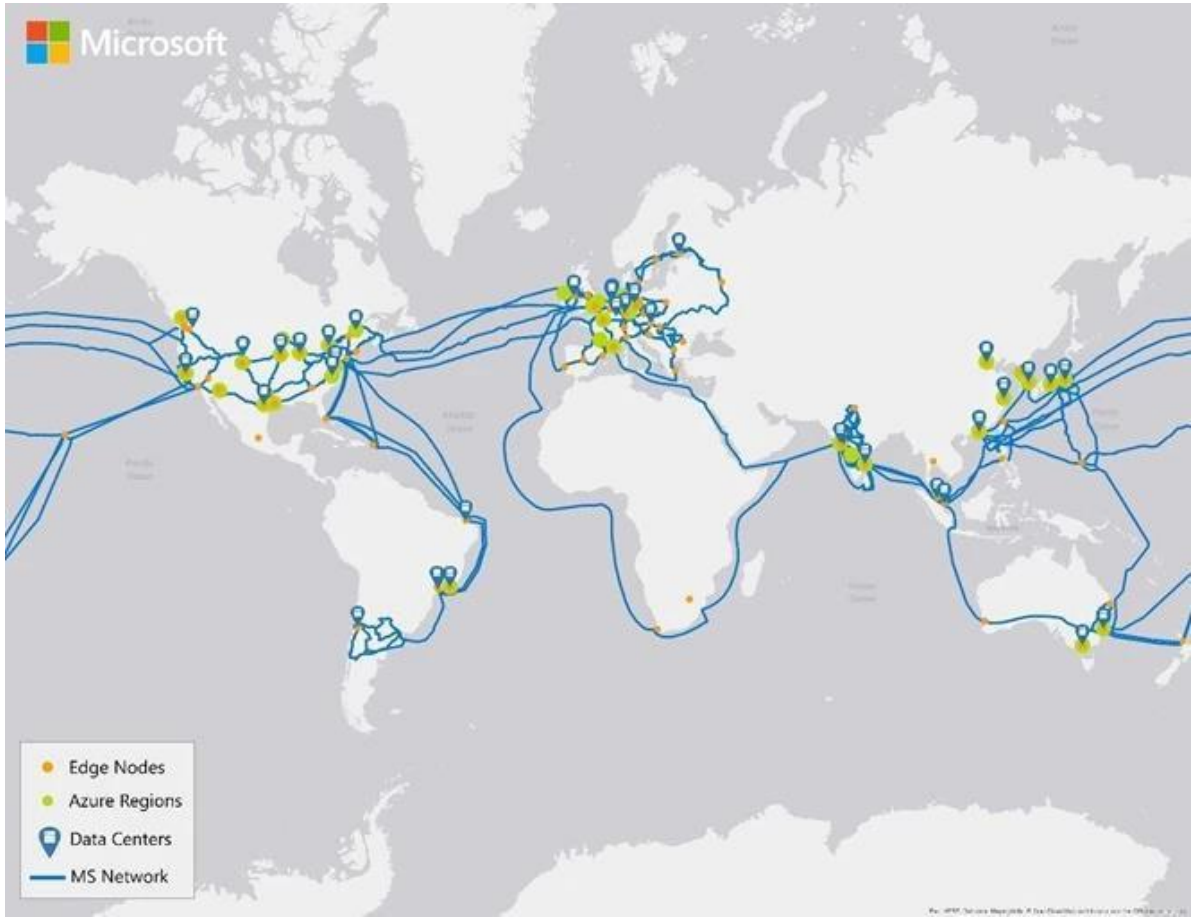
VPC to end user



Cloud providers have many datacenters and There are many layers to their networks

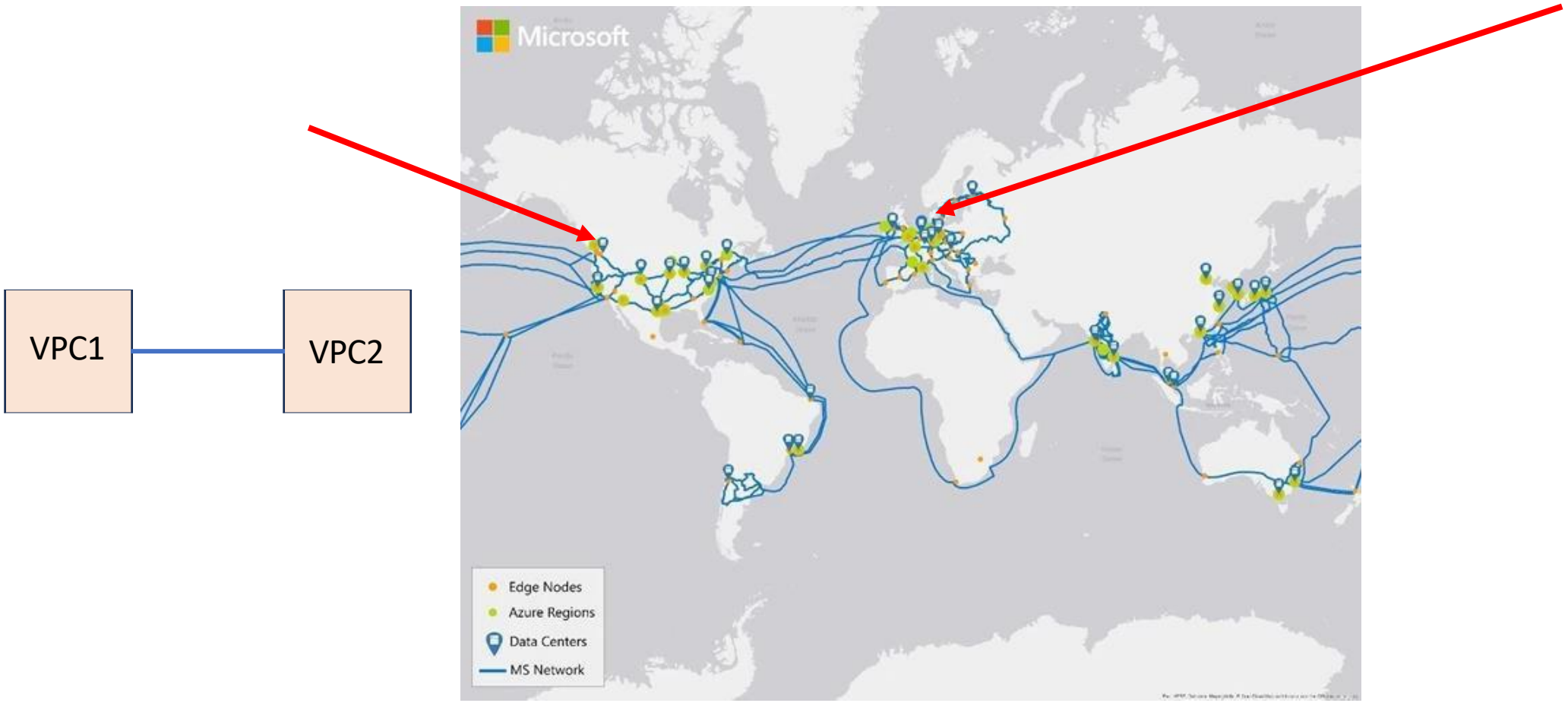


Microsoft’s Backbone Network



https://azure.microsoft.com/en-us/blog/how-microsoft-builds-its-fast-and-reliable-global-network/

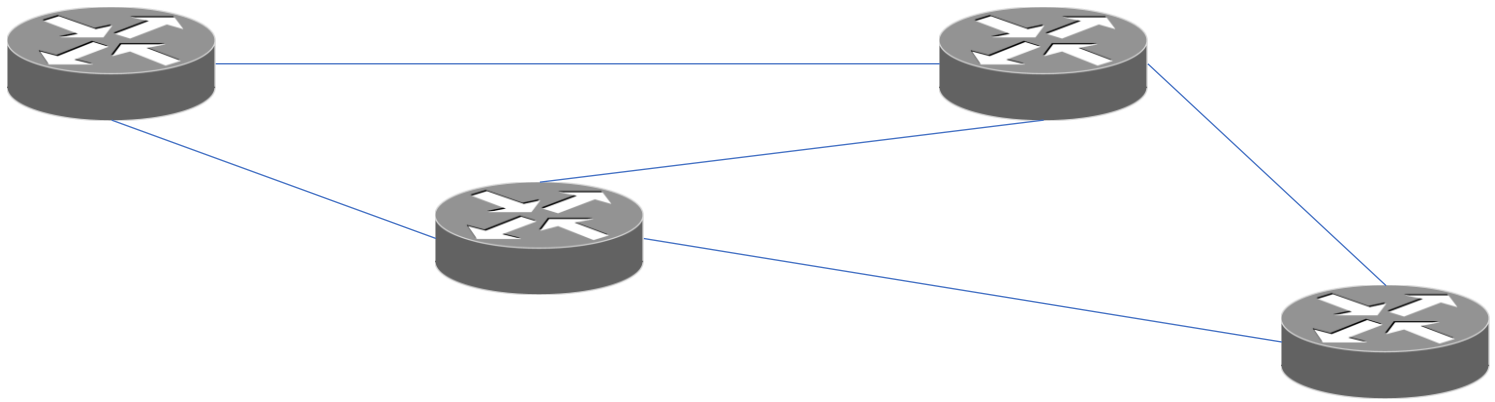
Microsoft's Backbone Network



https://azure.microsoft.com/en-us/blog/how-microsoft-builds-its-fast-and-reliable-global-network/

Can we just use standard routers (and protocols, and traffic engineering)?

- As we saw with datacenter networks… there is room for improvement



SWAN

- Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill Mohan Nanduri, Roger Wattenhofer. Achieving High Utilization with Software-Driven WAN. SIGCOMM 2013.
- [switch to their slides]
- Another good paper (for Google):
  - Sushant Jain, et. al. B4: experience with a globally-deployed software defined WAN. SIGCOMM 2013.



University of Colorado **Boulder**

Espresso

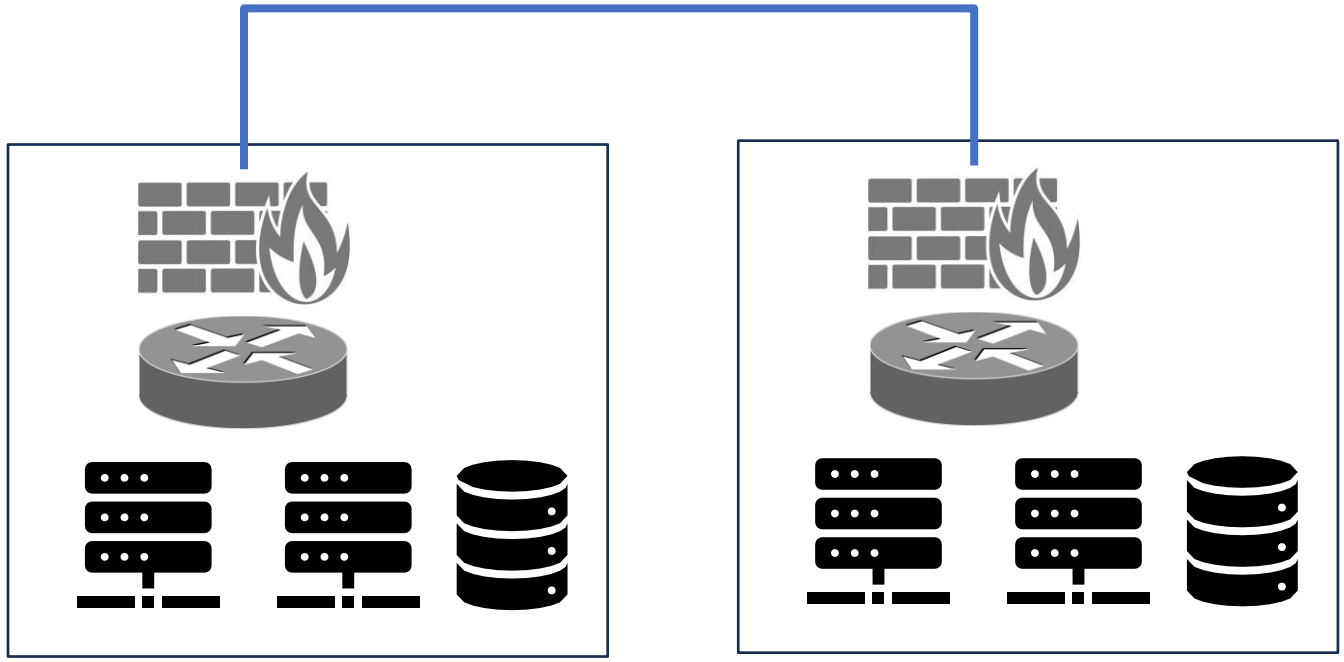
Course: Networking Principles in Practice – Cloud Networking
Module: WAN



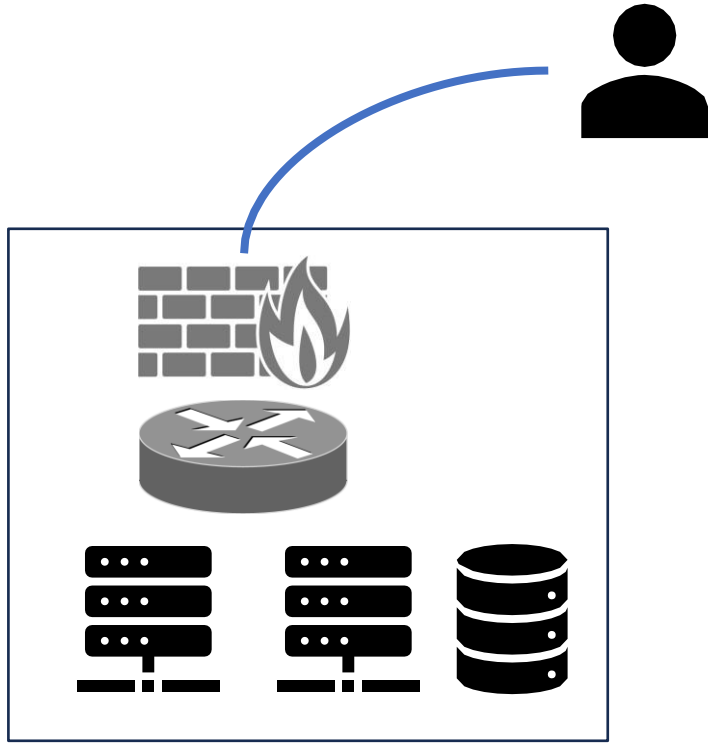
University of Colorado **Boulder**

Communication Beyond the VPC Bounds

VPC to VPC



VPC to end user



Google's Espresso

- KK Yap, et. al., Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering. SIGCOMM 2017.
- [switch to their slides]

- Also worth reading:
  - Angela Chiu, et. al., EdgePlex: Decomposing the Provider Edge for Flexibilty
  - and Reliability. SOSR 2015.



University of Colorado **Boulder**

Lab overview

Course: Networking Principles in Practice – Cloud Networking
Module: WAN

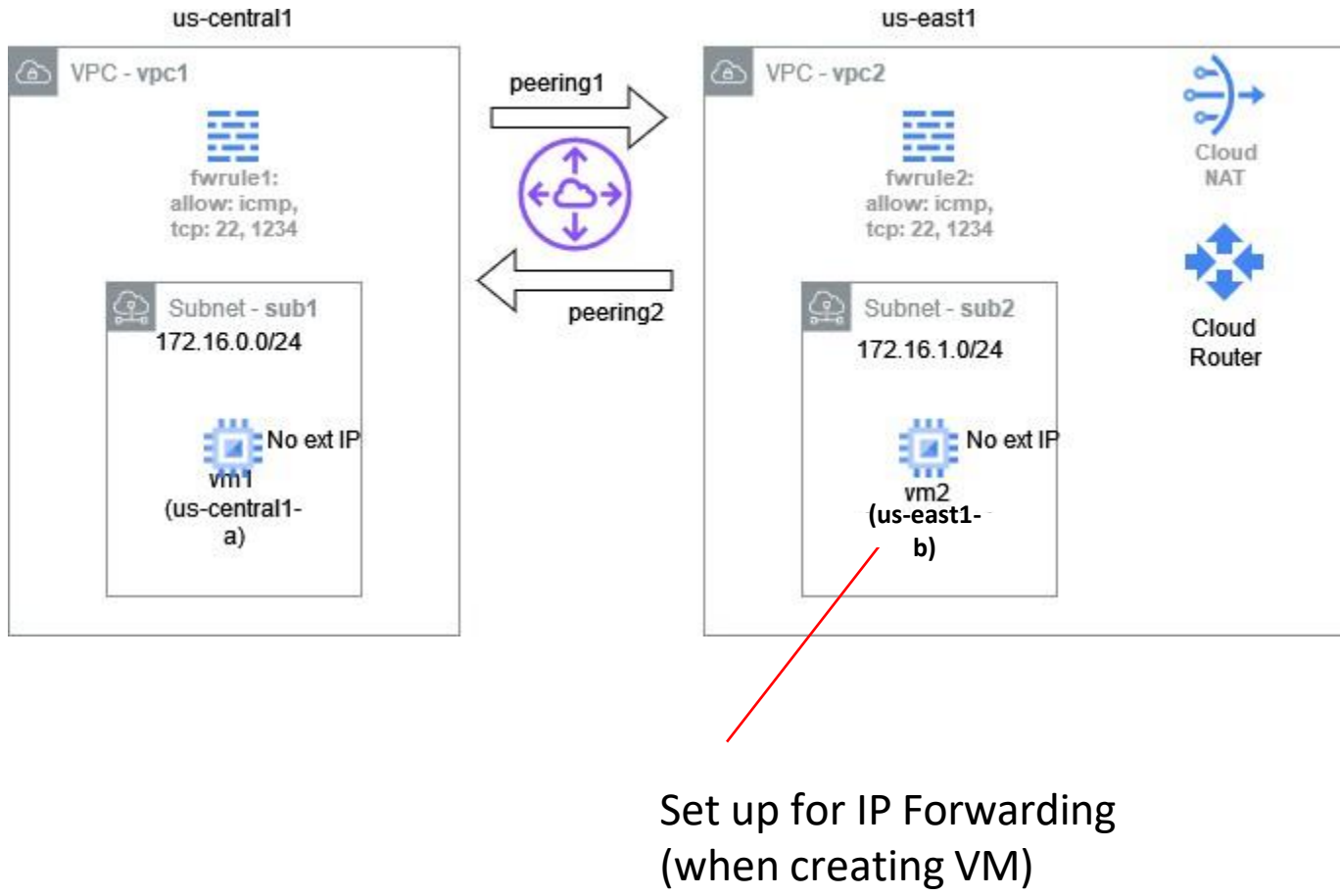


University of Colorado **Boulder**

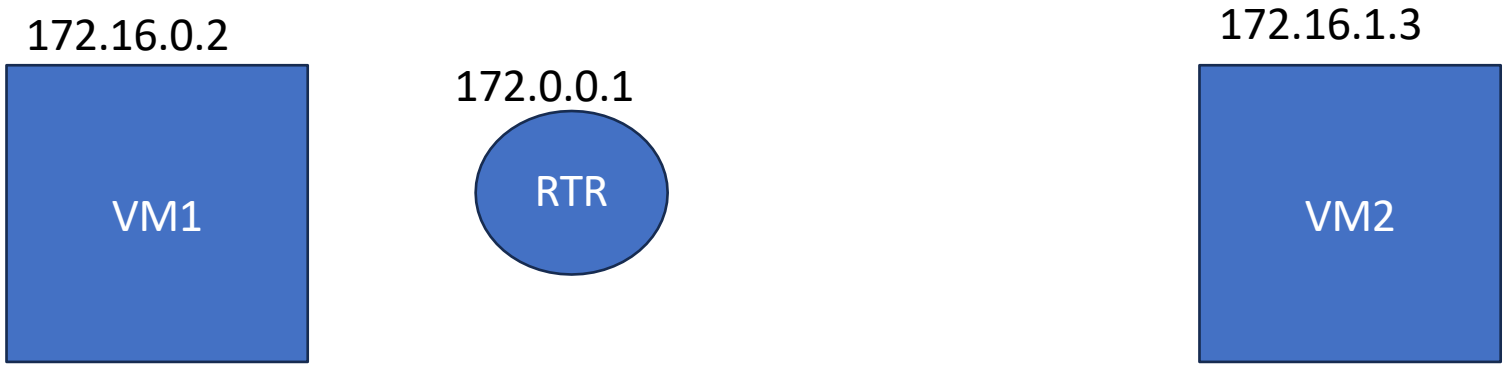
What you'll build (in terraform)

- VPC peering
- Cloud NAT
- Cloud router

- Then: Set up so traffic from vm1 can go through vm2 out to the Internet (can do manually)



Can’t just forward via VM2



X ip route add 34.223.124.45/32 via 172.16.1.2
(wget 34.223.124.45
will ARP for 172.16.1.2 – which will get no response)

Note: From VM2 you can do ping neverssl.com to make sure the IP address is still that. If not, change the IP address.

Need to Tunnel



> sudo ip link add vxlan0 type vxlan id 5001 local 172.16.0.2 remote 172.16.1.2 dev ens4 dstport 50000

> sudo ip addr add 192.168.100.2/24 dev vxlan0

> sudo ip link set up dev vxlan0

(will add a routing table entry for 192.168.100.0/24)

> sudo ip link add vxlan0 type vxlan id 5001 remote 172.16.0.2 local 172.16.1.2 dev ens4 dstport 50000

> sudo ip addr add 192.168.100.3/24 dev vxlan0

> sudo ip link set up dev vxlan0

(will add a routing table entry for 192.168.100.0/24)

Still won’t work: VM2 doesn’t know what to do



> sudo ip route add 34.223.124.0/24 via 192.168.100.3

> wget 34.223.124.45

> sudo tcpdump -i ens4 -n port not ssh

or

> sudo tcpdump -i vxlan0 -n

Note: From VM2 you can do ping neverssl.com to make sure the IP address is still that. If not, change the IP address.

Add NAT on VM2



> sudo vi /etc/sysctl.conf
    // uncomment this line: "net.ipv4.ip_forward=1"
    // (or add it if it doesn't exist)

> sudo sysctl -p

> sudo iptables -t nat -A POSTROUTING -s 192.168.0.0/16 -o ens4 -j MASQUERADE

Add NAT on VM2



> ip route add 34.223.124.45/32 via 192.168.100.3
(may need to delete route if you were following along)

> wget 34.223.124.45

Assessment

- Run terraform to create infrastructure
- Update VMs with any configuration (can be done as part of startup script, or manual – startup script will need to lookup the assigned IP)
- Answer some questions on coursera



University of Colorado **Boulder**